

BÀI 2: KIM TỰ THÁP YÊU CẦU PHẦN MỀM (Số tiết: 5 tiết)

Mục tiêu bài học

- Hiểu về mô hình kim tự tháp yêu cầu áp dụng trong quy trình kỹ nghệ yêu cầu.
- Hiểu được một số quy trình phần mềm tổng thể như RUP, Agile&Scrum, và sự tác động của nó lên kim tự tháp yêu cầu.
- Có khả năng lựa chọn, cài đặt, và sử dụng được các công cụ quản lý yêu cầu để quản lý các yêu cầu sau khi phân tích hình thành. Biết cách thức quản lý yêu cầu, theo dõi dấu vết yêu cầu và quản lý các thay đổi về yêu cầu được hỗ trợ

Nội dung chính

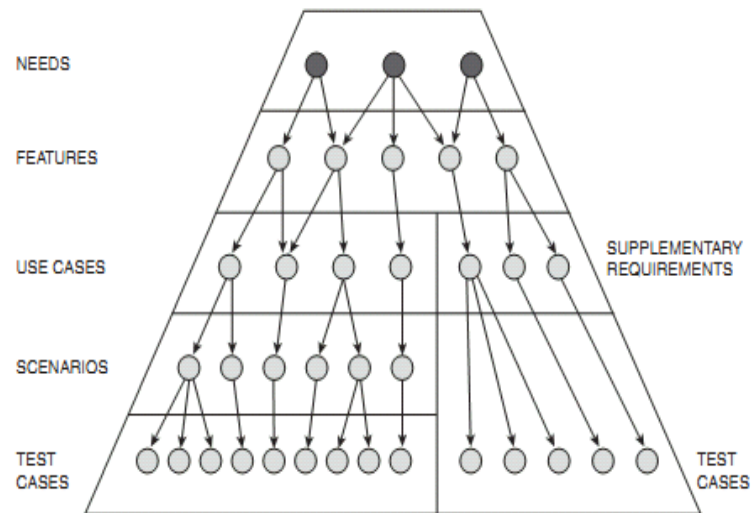
- Giới thiệu về mô hình kim tự tháp yêu cầu
- Kết hợp kim tự tháp yêu cầu với mô hình quy trình phần mềm cụ thể
- Giới thiệu một số công cụ quản lý yêu cầu
- Thảo luận

2.1 Giới thiệu mô hình kim tự tháp yêu cầu

Kim tự tháp yêu cầu phân chia các yêu cầu phần mềm thành bốn loại gồm: (1) NEEDs (Stakeholder's requests); (2) Product Features (FEATs); (3) Use cases (Ucs); và (4) Supplementary Requirements (SUPLs). Trong đó (1) & (2) được xem là các yêu cầu người dùng (user requirements) trong khi (3) & (4) được xem là các yêu cầu hệ thống (system requirements), viết cho developers triển khai phần mềm đáp ứng tập yêu cầu này.

Hai tầng, Scenarios & Test cases là các tầng bổ sung làm minh chứng cho khả năng test, khả năng nghiệm vụ của yêu cầu đã thu thập, phân tích xác định ở các mức trên.

Hình 2.1 Chỉ ra mô hình kim tự tháp yêu cầu



Hình 2.1 Kim tự tháp yêu cầu

Các tầng yêu cầu trong kim tự tháp bao gồm:

1. **Stakeholder need:** là một yêu cầu/ nhu cầu có nguồn gốc từ stakeholders
2. **Feature:** là một tính năng được cung cấp bởi hệ thống, thường được hình thành từ việc phân tích nghiệp vụ; mục đích của tính năng là hỗ trợ đầy đủ một nhu cầu của stakeholder (need)
3. **Use case:** là một mô tả về hành vi của hệ thống theo các thuật ngữ về chuỗi các hành động
4. **Supplementary requirement:** là một yêu cầu khác (thường là các yêu cầu phi chức năng) mà không đạt được trong các use cases
5. **Test case:** là một đặc tả về các đầu vào test, các điều kiện thực thi, và các kết quả mong đợi
6. **Scenario:** là một chuỗi hành động cụ thể, một đường đi cụ thể thông qua use case.

Lưu ý:

- Các yêu cầu thuộc cùng tầng sẽ thuộc cùng một loại. Các yêu cầu càng ở tầng thấp, mức độ chi tiết, chính xác càng cao.
- Từ 1 yêu cầu mức trên, có thể ánh xạ thành nhiều yêu cầu mức dưới. Các kiểu ánh xạ bao gồm: 1-1, 1-n, n-m. Ví dụ: Tài liệu vision mô tả các features có thể chỉ 12 trang, trong khi tài liệu đặc tả chi tiết (UC specifications) chứa các use case mức dưới có thể lên đến 200 trang.

Với mô hình kim tự tháp yêu cầu, quy trình RE được chia thành các giai đoạn:

1. **Thiết lập bản kế hoạch quản lý yêu cầu** (Establishing an RM Plan)
2. **Thu thập yêu cầu** (Requirements elicitation)
3. **Phát triển tài liệu tầm nhìn dự án** (Developing the Vision document)
4. **Tạo các Ucs** (Creating use cases)
5. **Đặc tả bổ sung** (Supplementary specification)
6. Tạo các test cases từ các use cases (Creating test cases from use cases (Testers được hỗ trợ bởi BAs))
7. Tạo các test case từ yêu cầu bổ sung (Creating test cases from supplementary requirements)

=> Các yêu cầu & tài liệu tương ứng sẽ được tạo ở mỗi bước (xem Bảng 2.1)

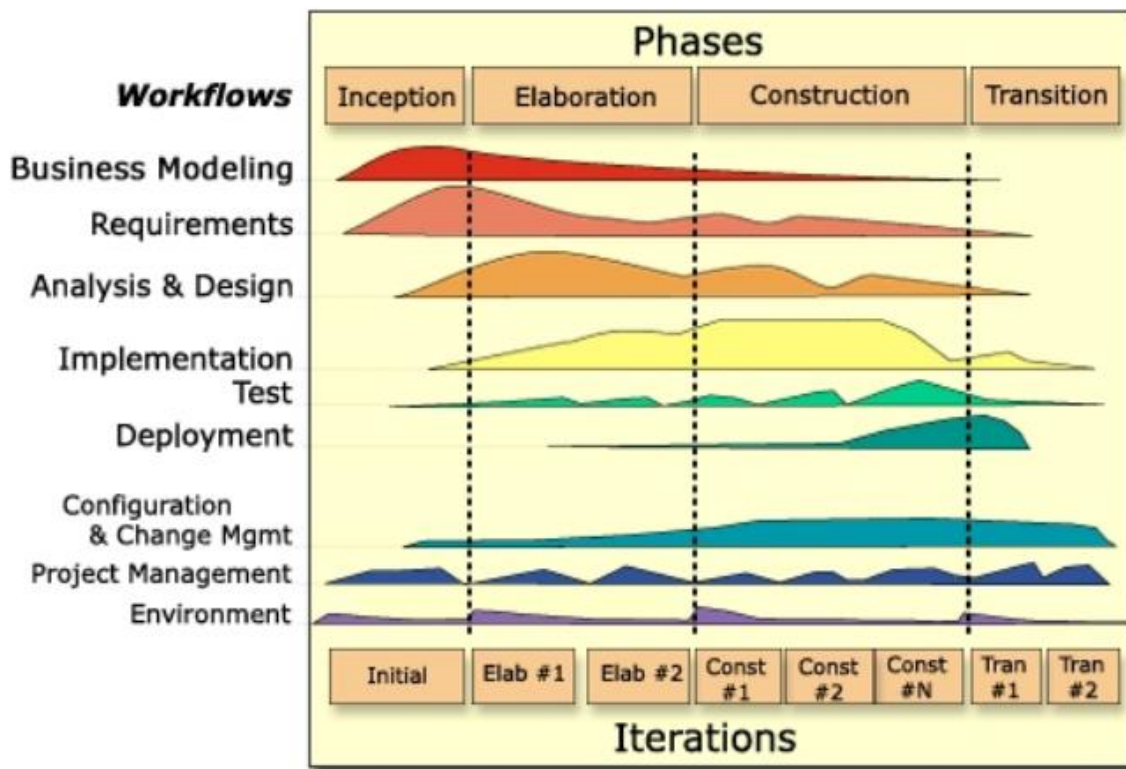
Bảng 2.1 Các bước triển khai trong kim tự tháp yêu cầu

Step	Requirement Types	Documents
Requirements elicitation	Stakeholder needs	Stakeholder requests
Developing the Vision document	Features	Vision
Creating use cases	Use cases, scenarios	Use case specifications
Supplementary specification	Supplementary requirements	Supplementary specification
Creating test cases from use cases	Test cases	Test cases
Creating test cases from the supplementary specification	Test cases	Test cases

2.2 Kết hợp kim tự tháp yêu cầu với mô hình quy trình phần mềm cụ thể

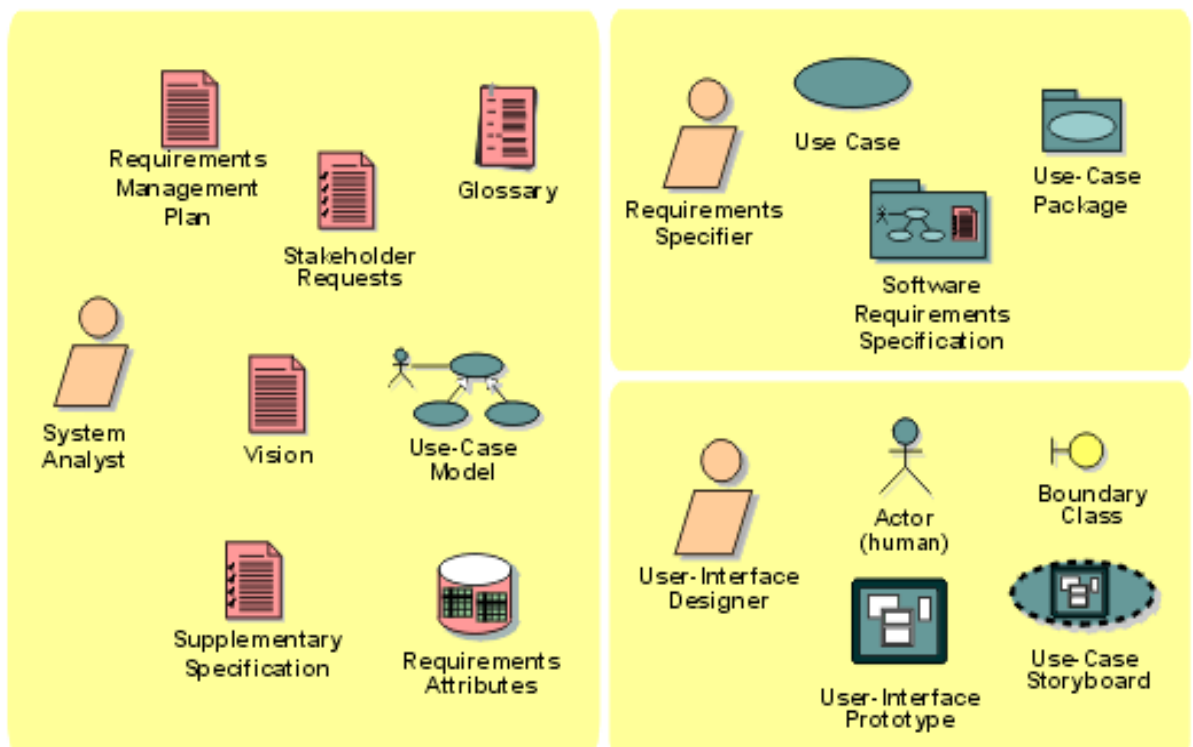
Ví dụ kết hợp với RUP (Rational Unified Process)

⇒ Hình 2.2 biểu diễn mô hình RUP



Hình 2.2 RUP

⇒ Kết hợp giữa RUP và kim tự tháp yêu cầu yêu cầu BA cần phân tích và hoàn thành các tài liệu yêu cầu như chỉ ra trong Hình 2.3



Hình 2.3 Vai trò và Tài liệu yêu cầu cần xây dựng

Trong đó:

- ❖ Với vai trò là phân tích viên hệ thống (System Analysis), bạn cần xây dựng:
 - Bản kế hoạch quản lý yêu cầu (Requirements Management Plan): mô tả cách tiếp cận tổng quát để phân tích và quản lý yêu cầu cho dự án cụ thể. Ở đây chúng ta tiếp cận theo mô hình kim tự tháp yêu cầu.
 - Kết quả khảo sát được lưu trong tài liệu với tên Stakeholders Requests Document (chứa các Needs được đề xuất bởi stakeholders – tầng 1 trong kim tự tháp yêu cầu)
 - Xây dựng tài liệu tầm nhìn dự án (Vision) chứa các tính năng sản phẩm (feature – tầng thứ 2 trong kim tự tháp yêu cầu)
 - Xây dựng tài liệu đặc tả bổ sung (Supplementary Specification) chứa các yêu cầu bổ sung, hay các yêu cầu phi chức năng của hệ thống – tầng 3 trong mô hình kim tự tháp yêu cầu
 - Xây dựng mô hình Use case và xác định các thuộc tính cần quản lý của từng kiểu yêu cầu.
- ❖ Với vai trò là người đặc tả yêu cầu (Requirements Specifier), bạn cần:
 - Xây dựng các đặc tả use case và xác định các gói use case để hình thành nên các thành phần trong kiến trúc hệ thống.
- ❖ Bên cạnh đó nhóm BA cần làm việc với nhà thiết kế giao diện phần mềm để thu thập và đặc tả các mẫu thử giao diện được đề xuất bởi người dùng.

❖ **Thảo luận:**

- Mô hình kim tự tháp yêu cầu + mô hình quy trình phần mềm thác nước?
- Mô hình kim tự tháp yêu cầu + mô hình quy trình Agile/Scrum?

2.3 Công cụ quản lý yêu cầu

Công cụ quản lý yêu cầu phần mềm giúp theo dõi, kiểm soát và quản lý các yêu cầu trong suốt vòng đời phần mềm. Các yêu cầu cần thiết đối với một công cụ RM:

1. Quản lý yêu cầu (Requirement Management)
2. Liên kết và Truy vết (Traceability)

3. Quản lý ưu tiên và phân loại yêu cầu
4. Quản lý các thay đổi (Change Management)
5. Quản lý tình trạng yêu cầu (Status Management)
6. Tính năng cộng tác (Collaboration)
7. Tích hợp với các công cụ khác
8. Báo cáo và phân tích (Reporting and Analytics)
9. Quản lý tài liệu (Document Management)
10. Bảo mật và quyền truy cập (Security and Access Control)
11. Tính năng hỗ trợ di động (Mobile Support)

=> Đây cũng là các tiêu chí cần quan tâm khi lựa chọn công cụ RM.

Một số công cụ RM phổ biến gồm:

1. JIRA (Atlassian)
2. IBM Engineering Requirements Management DOORS (DOORS Next)
3. Modern Requirements (for Azure DevOps)
4. Rational RequisitePro (IBM)
5. GitHub
6. AWS Jira Integration (Với Atlassian Jira)

Việc lựa chọn công cụ phù hợp cần căn cứ vào thực trạng dự án triển khai và các tính năng mà công cụ hỗ trợ.

❖ Jira

Jira bao gồm các tính năng chính như:

- Quản lý yêu cầu: JIRA cho phép tạo, phân loại và theo dõi yêu cầu phần mềm với các thông tin chi tiết như mức độ ưu tiên, trạng thái, và lịch sử thay đổi.
- Truy vết và liên kết: Hỗ trợ truy vết yêu cầu từ giai đoạn phân tích đến triển khai và kiểm thử, dễ dàng liên kết yêu cầu với các tác vụ, lỗi và mã nguồn.
- Quản lý thay đổi: Cung cấp tính năng theo dõi thay đổi yêu cầu, cho phép tạo các yêu cầu thay đổi.
- Quản lý ưu tiên: Giúp phân loại yêu cầu theo mức ưu tiên, giúp nhóm phát triển tập trung vào các yêu cầu quan trọng.
- Tích hợp mạnh mẽ: Tích hợp với các công cụ khác như Confluence (quản lý tài liệu), Bitbucket (quản lý mã nguồn), và các công cụ kiểm thử.

- **Báo cáo và phân tích:** cho phép xuất báo cáo chi tiết về tiến độ yêu cầu, hiệu suất nhóm và trạng thái của các yêu cầu trong dự án.
- **Cộng tác:** Hỗ trợ tính năng cộng tác với khả năng ghi lại notes, bình luận và thảo luận về các yêu cầu.

JIRA dùng phổ biến trong các môi trường Agile và Scrum. Tuy nhiên, nó có giao diện phức tạp, có thể cần tùy chỉnh để phù hợp với dự án cụ thể.

❖ **DOORS Next (IBM)**

Các tính năng nổi bật của DOOR NEXT bao gồm:

- **Quản lý yêu cầu:** cho phép tạo, phân loại yêu cầu, hỗ trợ các yêu cầu chức năng, phi chức năng, và yêu cầu bảo mật.
- **Truy vết và liên kết:** cho phép tạo mối quan hệ giữa các yêu cầu, mã nguồn, thiết kế và kiểm thử, đảm bảo truy vết đầy đủ xuyên suốt dự án.
- **Quản lý thay đổi:** cung cấp hệ thống kiểm soát thay đổi, cho phép theo dõi, phê duyệt các yêu cầu thay đổi.
- **Quản lý ưu tiên:** cung cấp các công cụ để phân loại và ưu tiên các yêu cầu theo mức độ quan trọng hoặc theo yêu cầu khách hàng.
- **Báo cáo và phân tích:** có khả năng tạo báo cáo chi tiết, các thống kê về yêu cầu, trạng thái và tiến độ của các yêu cầu trong dự án.
- **Cộng tác:** cho phép các nhóm làm việc cùng nhau, ghi chú và thảo luận về các yêu cầu trong thời gian thực.

DOOR là một trong những công cụ mạnh mẽ và lâu đời trong ngành công nghiệp phần mềm, nó hỗ trợ quản lý yêu cầu cho các dự án quy mô lớn và phức tạp, đặc biệt trong các ngành công nghiệp như hàng không, ô tô, và quốc phòng. Giao diện người dùng của DOORs có thể cứng nhắc và yêu cầu một quá trình học hỏi, chi phí sử dụng cao.

❖ **Modern Requirements (for Azure DevOps)**

Công cụ này có các tính năng chính bao gồm:

- **Quản lý yêu cầu:** Công cụ này tích hợp trực tiếp với Azure DevOps, cho phép quản lý và theo dõi yêu cầu từ giai đoạn đầu đến khi hoàn thành.
- **Truy vết và liên kết:** giúp người dùng kết nối yêu cầu với các user stories, task, và bug trong Azure DevOps, đảm bảo rằng yêu cầu được triển khai đầy đủ.

- **Quản lý thay đổi:** linh hoạt trong quản lý các yêu cầu thay đổi và theo dõi lịch sử thay đổi.
- **Báo cáo và phân tích:** Cho phép tạo báo cáo và tùy chỉnh, và công cụ phân tích giúp theo dõi tiến độ và quản lý dự án hiệu quả.
- **Cộng tác:** Hỗ trợ cộng tác giữa các thành viên nhóm thông qua tính năng ghi chú và bình luận trong các yêu cầu.

Modern Requirements có lợi thế dễ dàng tích hợp với Azure DevOps, một trong những công cụ quản lý dự án phổ biến nhất, thích hợp cho các nhóm phát triển sử dụng DevOps và Agile. Tuy nhiên, có thể không phù hợp với các tổ chức không sử dụng Azure DevOps hoặc các dự án không liên quan đến môi trường Microsoft.

❖ Rational RequisitePro (IBM)

Các tính năng chính của công cụ này gồm:

- **Quản lý yêu cầu:** tổ chức và quản lý yêu cầu phần mềm với nhiều cấp độ phân loại và nhóm yêu cầu.
- **Truy vết và liên kết:** hỗ trợ khả năng truy vết các yêu cầu.
- **Quản lý thay đổi:** giúp theo dõi và phê duyệt yêu cầu thay đổi.
- **Báo cáo và phân tích:** cho phép xuất báo cáo, thống kê chi tiết về yêu cầu để hỗ trợ quản lý dự án.
- **Cộng tác:** cho phép các nhóm làm việc, thảo luận trên cùng một yêu cầu.
 - o Rational RequisitePro khá phù hợp cho việc quản lý yêu cầu trong các dự án lớn và phức tạp, đặc biệt trong các lĩnh vực như y tế, hàng không, và quốc phòng. Nó hỗ trợ cách tạo dự án theo mô hình kim tự tháp yêu cầu (icon của công cụ), đồng thời hỗ trợ phát triển phần mềm theo quy trình phần mềm RUP. Tuy nhiên nhược điểm là không phù hợp với các dự án quy mô nhỏ hoặc đơn giản. Đây là công cụ mất phí, là ứng dụng Desktop.

❖ GitHub, AWS Jira Integration (Với Atlassian Jira)

- Sinh viên tham khảo thêm tại: <https://www.youtube.com/watch?v=4f8pj9IoVoA>

Thực hành:

SV thực hành: Cài đặt Rational RequisitePro, làm quen với công cụ: tạo dự án, nhập/sửa/xóa/tìm kiếm/truy vấn/... các thông tin về yêu cầu.

1. Nhận xét, đánh giá về Rational RequisitePro
2. SV đăng ký tài khoản trên GitHub, thực hiện tạo kho code, thêm issues, tạo pull requests, và quản lý dự án triển khai các yêu cầu đã cập nhật, theo dõi thay đổi, thảo luận, lọc thông tin về yêu cầu.

Lựa chọn tool RM phù hợp với dự án, cài đặt, làm quen với công cụ/dịch vụ lựa chọn.